

L'identité du token comme signal de routage pour des experts MLP résiduels

Anonymous authors

Paper under double-blind review

Abstract

Les routeurs Mixture-of-Experts appris utilisent les états cachés contextuels pour choisir quels paramètres traitent chaque token. Nous étudions la part de ce signal qui peut provenir de la seule identité du token lorsque la capacité experte forme un petit résiduel au-dessus d'un MLP dense partagé. Notre modèle associe chaque ID à deux experts étroits par une table fixe propre à chaque couche ; ces experts transforment le même état caché contextuel que la branche partagée. Dans une comparaison à graine unique entre deux modèles de 306,5M paramètres entraînés sur 8B tokens FineWeb-Edu, le modèle routé atteint une NLL sur le flux d'évaluation de 2,9329 contre 2,9482 pour la baseline dense au dernier checkpoint d'évaluation commun. Ce flux provient du split d'entraînement et l'implémentation routée est plus lente. Les checkpoints finaux restent proches sur ARC-Easy, PIQA et HellaSwag en zero-shot, tandis que le modèle routé obtient une perplexité WikiText-2 modestement inférieure. Un diagnostic séparé de routeur appris inverse son avantage au budget court face au dense à un budget apparié de 1,0003B tokens. La réplication multi-seed reste ouverte.¹

1 Introduction

Les modèles Mixture-of-Experts apprennent généralement un routeur à partir de l'état caché contextuel de chaque token (Shazeer et al., 2017; Fedus et al., 2022; Jiang et al., 2024). Ce routeur adapte la sélection au contexte, mais ajoute aussi des paramètres, des objectifs d'équilibrage et des décisions de dispatch. Un routage fixe fournit un contrepoint utile : si un signal simple au niveau du token alloue correctement les paramètres, certains gains attribués au routage contextuel peuvent provenir d'une capacité conditionnelle stable.

Nous testons cette idée avec une architecture volontairement asymétrique. Chaque token traverse un MLP SwiGLU dense partagé. Son identité sélectionne deux experts supplémentaires, beaucoup plus étroits, dont la sortie est ajoutée comme résiduel avec un gate appris. La sélection ne dépend pas du contexte, mais le calcul des experts en dépend : les deux branches reçoivent l'état caché produit par le transformer. L'expérience demande donc si l'identité du token sélectionne utilement un sous-espace de paramètres résiduels, et non si un lookup fixe par ID de token peut remplacer le calcul contextuel.

La comparaison principale utilise deux modèles de 306,5M paramètres entraînés avec le même budget de 8B tokens. Ils partagent leur backbone et leur largeur MLP stockée : l'un utilise une largeur dense de 4 096, l'autre une largeur partagée de 3 840 et quatre experts de largeur 64, dont deux actifs par token. Au dernier point du flux d'évaluation fixe, le modèle routé atteint 2,9329 contre 2,9482. Il s'entraîne plus lentement dans l'implémentation actuelle, et le flux d'évaluation provient du split train FineWeb-Edu.

Cette étude apporte une observation contrôlée, pas un verdict général sur le routage d'experts. Elle définit une architecture shared-plus-residual où l'identité du token contrôle uniquement la sélection, vérifie les tables réalisées et le comptage de capacité, puis rapporte la trajectoire complète à tokens appariés et une évaluation

¹Des outils d'IA générative ont été utilisés comme assistants généraux pour la révision linguistique, les contrôles de cohérence entre le manuscrit et les fichiers de métriques CSV/JSON fournis par les auteurs, ainsi que les scripts de figures et de révision. Les auteurs ont relu et vérifié l'ensemble des résultats expérimentaux, valeurs numériques, affirmations scientifiques et le texte final, et assument l'entière responsabilité de la soumission.

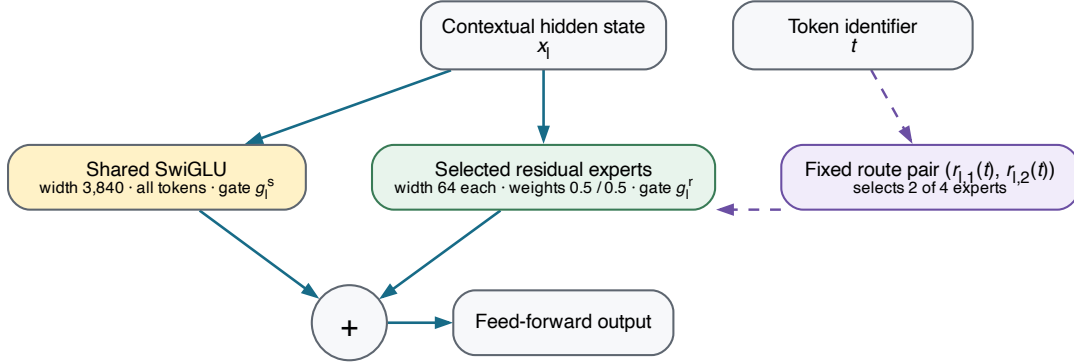


Figure 1: L’identité du token contrôle uniquement le lookup top-2 fixe. La branche partagée et les experts résiduels sélectionnés reçoivent le même état caché contextuel.

zero-shot des checkpoints finaux. Nous rapportons aussi un diagnostic de routeurs contextuels avec contrôles dense et fixe par identité de token appariés au budget court, suivi d’une comparaison routeur appris/dense à 1B tokens.

2 Travaux connexes

Les routeurs MoE appris utilisent les états cachés pour sélectionner les experts et ajoutent souvent un objectif auxiliaire ou un autre mécanisme d’équilibrage (Shazeer et al., 2017; Fedus et al., 2022; Jiang et al., 2024). Ces systèmes testent la valeur d’une spécialisation adaptée au contenu. Notre expérience teste un autre axe : une identité de token stable est-elle déjà informative lorsque le routage ne contrôle qu’une branche résiduelle étroite ?

Hash Layers est le précédent le plus proche (Roller et al., 2021). Ce travail montre que des hash fixes de caractéristiques locales des tokens peuvent concurrencer un routage appris. Notre architecture maintient un MLP dense actif pour chaque token et applique le lookup top-2 fixe à une petite capacité résiduelle. Le chemin routé corrige ainsi un calcul commun plutôt que de constituer la transformation feed-forward principale.

Des experts partagés apparaissent aussi dans DeepSeekMoE (Dai et al., 2024), où ils capturent le calcul commun à tous les tokens. Des analyses statistiques des mélanges à experts partagés fournissent des résultats sous des hypothèses d’estimation explicites (Nguyen et al., 2025). Nous utilisons cette décomposition comme prior architectural ; nos mesures ne testent pas ces garanties théoriques.

3 Architecture

3.1 Vue d’ensemble

L’architecture ne modifie que la sous-couche feed-forward d’un transformer decoder-only. L’attention reste une GQA standard à 16 têtes de requête et quatre têtes clé/valeur, avec QK-Norm, RoPE et attention causale SDPA. La Figure 1 isole le calcul modifié. Pour l’ID t et l’état caché contextuel $\mathbf{x}$, l’ID sélectionne les paramètres résiduels tandis que les deux branches transforment $\mathbf{x}$.

3.2 Routage fixe par identité de token

Chaque couche stocke une table top-2 fixe. L’expert primaire est obtenu par une permutation seedée π_l du résidu de l’ID :

$$r_{l,1}(t) = \pi_l(t \bmod E), \quad E = 4. \quad (1)$$

La table secondaire est construite une fois, par ordre croissant d’ID, en choisissant l’expert le moins utilisé parmi ceux distincts du primaire :

$$r_{l,2}(t) = \arg \min_{e \neq r_{l,1}(t)} \sum_{u=0}^{t-1} \mathbb{1}[r_{l,2}(u) = e], \quad (2)$$

avec départage par l’indice le plus faible. Comme le vocabulaire de 32 000 tokens est divisible par quatre, chaque expert reçoit exactement 8 000 IDs primaires. Cela équilibre la cardinalité du vocabulaire, pas le trafic corpus.

3.3 MLP partagé et experts résiduels

Pour l’état caché contextuel $\mathbf{x}$ et l’ID t , la sortie feed-forward est

$$\text{MLP}_l(\mathbf{x}, t) = g_l^s \text{Shared}_l(\mathbf{x}) + g_l^r \sum_{j=1}^2 0.5 \text{Expert}_{r_{l,j}(t)}(\mathbf{x}), \quad (3)$$

où les gates scalaires appris g_l^s et g_l^r sont initialisés à 1,0 et 0,1. Les sorties expertes sélectionnées utilisent des poids fixes 0,5/0,5. Chaque branche est une transformation SwiGLU,

$$\text{Expert}_i(\mathbf{x}) = (\text{SiLU}(\mathbf{x}\mathbf{W}_{gate}^i) \odot \mathbf{x}\mathbf{W}_{up}^i)\mathbf{W}_{down}^i \quad (4)$$

et la branche partagée a la même forme avec ses propres paramètres. L’identité sélectionne des fonctions, mais ces fonctions opèrent sur des états contextuels. L’objectif de langage est appliqué après le réseau complet ; il n’existe pas d’objectif indépendant par expert.

3.4 Capacité et dispatch sparse

Soient d la dimension du modèle, d_s la largeur partagée, d_e la largeur de chaque expert, E le nombre d’experts stockés et k le nombre sélectionné par token. En ignorant les biais, le MLP routé stocke

$$P_{\text{TR}} = 3d(d_s + Ed_e), \quad (5)$$

paramètres par couche et évalue la largeur intermédiaire

$$d_{\text{active}} = d_s + kd_e. \quad (6)$$

par token. Pour le modèle évalué, $(d_s, E, d_e, k) = (3840, 4, 64, 2)$: la largeur stockée est 4 096, comme la baseline dense, et la largeur active est 3 968. L’implémentation groupe les tokens selon leurs assignations et n’évalue que les experts sélectionnés. Les petits kernels et le dispatch ajoutent malgré tout un surcoût mesuré en Section 5.1.

4 Protocole expérimental

4.1 Comparaison principale

Nous entraînons un modèle à résiduel token-identity et une baseline dense, tous deux de 306,5M paramètres. Ils utilisent le même backbone à 18 couches, le même tokenizer, un contexte de 2 048 tokens, le même flux FineWeb-Edu, le même optimiseur, le même scheduler, la précision BF16 et un budget de 8B tokens. Le batch global est de 1 048 576 tokens (huit GPUs, batch 64 par GPU) : un même nombre de steps implique donc un même nombre de tokens vus. La différence contrôlée est la paramétrisation MLP de la Table 1.

Table 1: Configurations appariées des modèles 300M.

Paramètre	Résiduel token-identity	Baseline dense
Couches (L)	18	18
Dimension (d_{model})	1024	1024
Têtes d’attention (n_h)	16	16
Têtes KV (n_{kv})	4 (ratio GQA 4:1)	4 (ratio GQA 4:1)
Dimension intermédiaire routée	256 total	—
Intermédiaire expert	64	—
Intermédiaire shared expert	3840	—
Experts ($n_{experts}$)	4, top- $k = 2$	1 (dense)
Initialisation gates shared/routed	1,0 / 0,1	—
Vocabulaire	32000	32000
Contexte max	2048	2048
Total paramètres	306,5M	306,5M
Chemin MLP actif/token	shared + 2 experts routés	SwiGLU dense

4.2 Optimisation

Les deux modèles utilisent AdamW avec $(\beta_1, \beta_2) = (0, 9, 0, 95)$, un weight decay de 0,1, un gradient clipping à 1,0, un warmup de 5% et une décroissance cosinus jusqu’à 10% du peak learning rate. Les projections de sortie résiduelles utilisent

$$\sigma_{\text{residual}} = \frac{0.02}{\sqrt{2 \times L}} \quad (7)$$

pour L couches (Radford et al., 2019).

4.3 Protocole de routage et d’évaluation

L’inspection des checkpoints montre que les buffers de lookup primaire et secondaire des 18 couches correspondent exactement aux équations 1–2. Nous identifions donc le mécanisme réalisé directement depuis les assignations sauvegardées : routage primaire modulo permuté et assignation secondaire équilibrée par cardinalité, plutôt que depuis le nom historique du run. Le flux d’évaluation fixe provient du split train FineWeb-Edu ; il est identique entre les deux runs mais ne constitue pas un holdout indépendant.

4.4 Protocole des benchmarks standard

Nous exportons les checkpoints finaux du step 7 629 vers le même runtime d’inférence BF16 et évaluons les deux modèles sans fine-tuning spécifique aux tâches. EleutherAI LM Evaluation Harness v0.4.12 exécute ARC-Easy, PIQA, HellaSwag et WikiText-2 sans limite d’exemples, avec des prompts zero-shot, un batching automatique, un contexte maximal de 2 048 et des graines identiques. Nous rapportons l’accuracy normalisée par longueur pour les trois tâches à choix multiples et la perplexité par mot pour WikiText-2. Un contrôle de génération apparié sur cinq prompts est exécuté avec CUDA Graph activé avant l’évaluation complète. Les commandes exactes, hashes des checkpoints, versions des paquets, métriques agrégées et le sanity record du runtime figurent dans les artefacts supplémentaires.

5 Résultats

5.1 Comparaison de scaling (300M, 8B tokens)

Le modèle à résiduel token-identity commence derrière la baseline dense et la croise après environ 0,8B tokens (Figure 2). Au dernier checkpoint d’évaluation commun, step 7 500 ou 7,864B tokens, sa NLL sur le flux d’évaluation est de **2,9329** contre **2,9482** pour dense, soit un écart de $-0,0153$. Au dernier point train

apparié, step 7 620, la NLL d’entraînement est de **2,9231** contre **2,9324**. La Table 2 identifie explicitement les deux métriques.

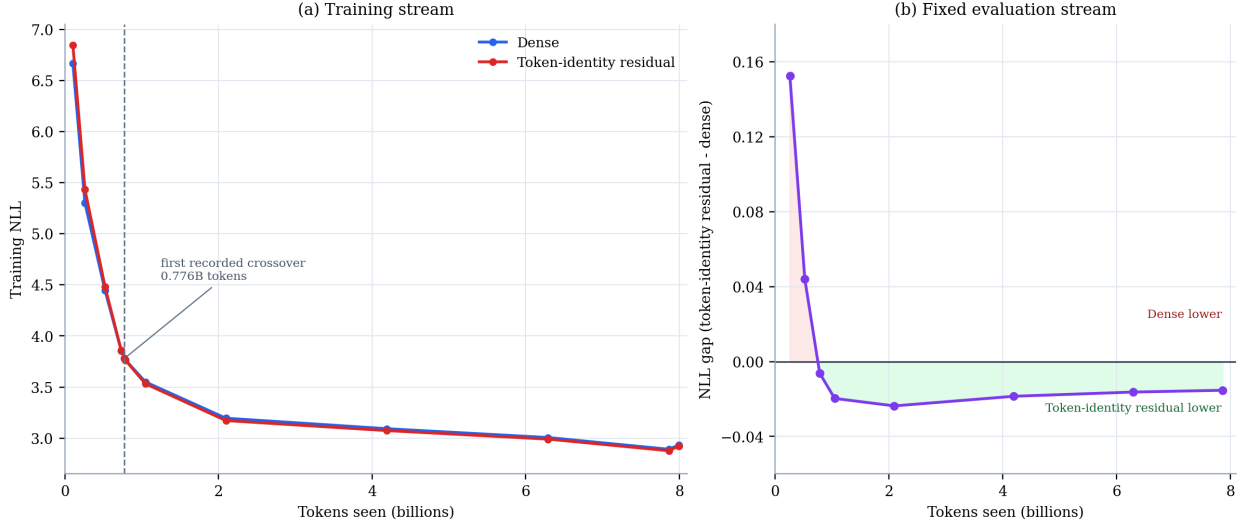


Figure 2: Mesures de perte à nombres de tokens appariés. À gauche : NLL d’entraînement aux checkpoints consignés. À droite : NLL du résiduel token-identity moins celle du modèle dense sur le flux fixe du split train FineWeb-Edu ; les valeurs négatives favorisent le résiduel token-identity. Les marqueurs sont les checkpoints consignés et les lignes servent de guide visuel. Le flux d’évaluation n’est pas held-out.

Table 2: NLL d’entraînement et du flux d’évaluation aux mêmes nombres de tokens. La ligne d’évaluation utilise le flux fixe du split train FineWeb-Edu ; les autres lignes sont des mesures d’entraînement.

Métrique	Step	Tokens	Dense	TR ($k = 2$)	Écart
Train	100	105M	6,6698	6,8464	+0,177
Train	500	524M	4,4450	4,4796	+0,035
Train	1000	1,05B	3,5500	3,5324	−0,018
Train	2000	2,10B	3,1953	3,1720	−0,023
Train	4000	4,19B	3,0925	3,0738	−0,019
Train	6000	6,29B	3,0061	2,9906	−0,015
Flux eval.	7500	7,864B	2,9482	2,9329	−0,0153
Train	7620	7,99B	2,9324	2,9231	−0,009

Trafic de routage. L’utilisation finale des experts est 0,2481/0,2635/0,2481/0,2403, sans expert mort. Il s’agit d’une propriété observée du run, pas d’une conséquence de l’équilibre de cardinalité du vocabulaire.

Throughput d’entraînement. Sur le même système 8×B300 et avec le même batch global, dense atteint environ 0,95M tokens/s et le modèle à résiduel token-identity environ 0,75M tokens/s. La comparaison de qualité est appariée par tokens, pas par temps mural.

5.2 Évaluation zero-shot des checkpoints 300M finaux

La Table 3 rapporte les runs complets, sans sous-échantillonnage. Le checkpoint routé est 0,46 point de pourcentage sous le dense sur ARC-Easy, à égalité à la précision affichée sur PIQA, 0,39 point au-dessus sur HellaSwag et obtient une perplexité WikiText-2 inférieure de 0,59. Chaque écart d’accuracy est inférieur à

une erreur standard rapportée : ce panneau soutient donc une quasi-parité, pas une affirmation de supériorité par tâche.

Table 3: Évaluation zero-shot des checkpoints 300M finaux appariés. ARC-Easy (2 376 exemples), PIQA (1 838) et HellaSwag (10 042) rapportent l’accuracy normalisée par longueur en pourcentage ; WikiText-2 rapporte la perplexité par mot (plus bas est meilleur).

Modèle	ARC-Easy	PIQA	HellaSwag	WikiText-2 PPL
Dense	49,07	63,49	33,67	35,79
Résiduel token-identity	48,61	63,49	34,06	35,20

5.3 Ablations 100M

Le panneau A contient sept modèles historiques d’environ 100M paramètres, chacun entraîné sur 1,0003B tokens avec une graine unique sur la configuration 2×B200 consignée. La suite sépare la branche partagée, la branche résiduelle et plusieurs assignations top-2 fixes. La condition configurée pour le routage frequency-aware ne contient aucune table de fréquences remplie et réalise donc un lookup modulo-primaire/secondaire équilibré. Les contrôles modulo et round-robin ont les mêmes assignations primaires dans ce protocole ; leurs différences sont descriptives, pas causales.

La différence entre les steps train et évaluation des panneaux suivants reflète le calendrier de logging enregistré, et non une substitution entre checkpoints. Les runs à 1B tokens ont un budget de 954 steps, consistent les métriques train tous les dix steps et sont évalués tous les 750 steps : le step 950 est donc leur dernier point train commun consigné et le step 750 leur dernier checkpoint d’évaluation commun. Les runs courts s’arrêtent au step 95, consistent les métriques train tous les cinq steps et sont évalués tous les 75 steps, ce qui produit la paire analogue step 95/step 75.

Table 4: Panneau A : ablations 100M historiques à budget apparié (2×B200, 1,0003B tokens). La perte d’évaluation utilise le flux fixe du split train FineWeb-Edu au step 750. Le débit n’est comparable qu’au sein de ce panneau.

Variante	Train @ 950	Eval. @ 750	Tok/s
Modulo-primaire/secondaire équilibré + shared	4,8205	4,8887	321k
Dense residual	4,8244	4,8958	397k
Modulo-adjacent top-2 + shared	4,8320	4,9016	321k
Round-robin top-2 + shared	4,8384	4,9072	321k
Random top-2 + shared	4,8875	4,9577	318k
Shared-only	4,9285	5,0012	402k
Modulo-primaire/secondaire équilibré, sans shared	4,9693	5,0393	334k

La branche partagée est importante dans cette suite : la retirer donne la perte d’évaluation la plus élevée, tandis que l’utiliser sans experts résiduels routés reste derrière les variantes dense et shared-plus-routed. Le run shared à secondaire équilibrée obtient la perte la plus basse, de peu devant dense. Chaque ligne restant single-seed et certains contrôles nominaux partageant la même assignation primaire, ces résultats comparent les composants mais n’établissent pas un classement causal des assignations fixes.

Pour tester le contrôle reviewer-relevant par routeur appris avant d’engager des runs full-budget, le panneau B utilise un shard mmap FineWeb-Edu tokenisé avec o200k et 99,6M tokens d’entraînement sur 2×RTX PRO 6000 Blackwell. Les quatre variantes utilisent la même graine, le même ordre de tokens, le même backbone de 98,2M paramètres, le même batch et le même scheduler. Les routeurs contextuels sélectionnent deux experts depuis les états cachés et n’utilisent aucune statistique de fréquence. Le contrôle fixe par identité de token utilise au contraire les comptages corpus exacts pour équilibrer sa route secondaire fixe.

Table 5: Panneau B : diagnostic court des routeurs appris (2×RTX PRO 6000 Blackwell, 99,6M tokens). Les valeurs ne sont comparables qu’au sein de ce panneau, pas avec le panneau A. La perte train rapportée est la perte de langage ; le terme auxiliaire est exclu.

Variante	Params	Train @ 95	Eval. @ 75
Top-2 contextuel appris + équilibrage auxiliaire	98,213M	7,4655	7,5647
Résiduel dense	98,197M	7,4734	7,5766
Top-2 contextuel appris + équilibrage loss-free	98,213M	7,5072	7,6052
Modulo-primaire/secondaire corpus-balanced + shared	98,197M	7,5310	7,6273

À ce budget court, l’équilibrage auxiliaire améliore la perte d’évaluation de 0,0119 face au dense et de 0,0626 face au contrôle fixe par identité de token. Son terme auxiliaire au step 95 vaut 0,0101, aucun expert n’est mort et son excès de paramètres face au contrôle fixe est de 15 360 paramètres (0,0156%). L’équilibrage loss-free reste derrière le dense. Ces écarts early-budget à graine unique ont motivé la promotion de la seule variante auxiliaire vers le budget complet. Le débit est omis car les panneaux A et B utilisent des matériels différents et le chemin de routage fixe public ne dispose pas du kernel sparse optimisé de l’implémentation historique.

Le run promu et son contrôle dense ont tous deux terminé 1,0003B tokens sur la même configuration o200k que le panneau B. Le panneau C compare cette paire séparément du panneau A.

Table 6: Panneau C : comparaison full-budget appariée du routeur appris avec équilibrage auxiliaire et du contrôle dense (2×RTX PRO 6000 Blackwell, 1,0003B tokens). Ce panneau n’est pas directement comparable au panneau A. La perte train exclut le terme auxiliaire.

Variante	Params	Train @ 950	Eval. @ 750	Tok/s
Top-2 contextuel appris + équilibrage auxiliaire	98,213M	4,7909	4,8580	95k
Résiduel dense	98,197M	4,7849	4,8433	100k

Au step 750, le trafic expert vaut 0,2400/0,2533/0,2610/0,2457, aucun expert n’est mort et le terme auxiliaire vaut 0,0100. Le contrôle dense obtient une perte d’évaluation inférieure de 0,0147 et un débit supérieur d’environ 5,2%. L’avantage de 0,0119 du routeur auxiliaire au budget court ne persiste donc pas à 1B tokens dans cette paire single-seed. Ce panneau compare le routage appris au calcul dense ; la réplication multi-seed reste ouverte.

6 Discussion et limitations

Le résultat montre qu’un ID fixe peut sélectionner un sous-espace de paramètres résiduels utile dans une paire de runs appariés. Il ne montre pas que les IDs définissent des classes expertes sémantiques. Les assignations sont des partitions modulo à une permutation propre à chaque couche près, et les experts sélectionnés transforment des états cachés contextuels. Une interprétation possible est qu’une capacité stable conditionnée par token modifie les fonctions disponibles pour les éléments lexicaux récurrents, tandis que le MLP partagé et l’attention conservent le traitement contextuel commun.

L’expérience n’isole pas non plus l’identité du token de toutes les explications alternatives. Les ablations 100M de la Section 5.3 incluent des contrôles résiduels, aléatoires et contextuels appris. Le panneau court favorise l’équilibrage auxiliaire, mais son contrôle dense apparié passe devant à 1B tokens tandis que le routeur promu termine sans expert mort. Les panneaux B et C utilisent une graine et la même famille de tokenizer o200k que le panneau A, mais un shard et un pipeline d’ingestion différents, un autre snapshot de l’implémentation et un autre matériel. Ils comparent donc le routage appris au calcul dense sur une configuration sans établir la supériorité générale du routage appris ou fixe. Les contrôles fixes historiques ne permettent pas non plus un classement causal des assignations car certains contrôles nominaux réalisent les mêmes assignations primaires.

La comparaison 300M utilise une graine par architecture et son flux fixe d'évaluation de perte vient du split train. Le panneau zero-shot standard est limité à trois tâches à choix multiples relativement simples et WikiText-2 ; ses écarts d'accuracy sont inférieurs à une erreur standard et il ne remplace ni une évaluation OOD plus large ni une réplification multi-seed. L'implémentation routée est plus lente que dense au même batch. Enfin, les buffers de lookup sauvegardés réalisent la règle modulo-primary/secondary-balanced par cardinalité, et non une assignation dépendant des fréquences. Ces limites préservent l'observation centrale mais en restreignent la portée.

7 Conclusion

L'identité du token fournit un signal de routage résiduel utile dans la paire de runs à 306,5M paramètres : après 8B tokens d'entraînement, le modèle shared-plus-routed termine devant une baseline dense appariée. L'écart observé de NLL sur le flux d'évaluation est de 0,0153 ; l'incertitude n'est pas estimée car chaque architecture n'utilise qu'une graine, et l'implémentation routée est plus lente en temps mural. Cette observation pose une question concrète aux systèmes MoE appris : quelle quantité de routage contextuel est nécessaire lorsqu'un chemin dense partagé traite déjà le calcul commun ?

Déclaration d'impact sociétal

Ce travail étudie une modification architecturale pour les modèles de langage. La soumission n'inclut pas les poids ; toute publication future devra respecter les exigences applicables de données, de sécurité et de licence.

Déclaration de reproductibilité

Le supplément fournit l'implémentation PyTorch, les tests de routage, les configurations réalisées, les métriques brutes et les scripts de figures. Il inclut le tokenizer 32k de l'expérience principale ainsi que la configuration o200k et le script de préparation du shard utilisés pour le diagnostic court des routeurs appris.

References

- Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, et al. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*, 2024.
- William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *The Journal of Machine Learning Research*, 23(1):5232–5270, 2022.
- Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- Huy Nguyen, Thong T. Doan, Quang Pham, Nghi D. Q. Bui, Nhat Ho, and Alessandro Rinaldo. On deepseekmoe: Statistical benefits of shared experts and normalized sigmoid gating. *arXiv preprint arXiv:2505.10860*, 2025.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 2019.
- Stephen Roller, Sainbayar Sukhbaatar, Arthur Szlam, and Jason Weston. Hash layers for large sparse models. *Advances in Neural Information Processing Systems*, 34:17555–17566, 2021.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.